

第一次培训

进制转换

二进制转十进制：位权相加之和

十六转二进制：十六进制每一位可以表示4位二进制数，因此可以将十六进制的每一位直接转换为对应的四位二进制数。

二进制转十六进制：每四位转换成一十

ASCII 值	16 进制	控制字符	ASCII 值	16 进制	控制字符	ASCII 值	16 进制	控制字符	ASCII	16 进制	控制字符
0	00H	NUT	32	20H	(space)	64	40H	@	96	60H	`
1	01H	SOH	33	21H	!	65	41H	A	97	61H	a
2	02H	STX	34	22H	"	66	42H	B	98	62H	b
3	03H	ETX	35	23H	#	67	43H	C	99	63H	c
4	04H	EOT	36	24H	\$	68	44H	D	100	64H	d
5	05H	ENO	37	25H	%	69	45H	E	101	65H	e
6	06H	ACK	38	26H	&	70	46H	F	102	66H	f
7	07H	BEL	39	27H	.	71	47H	G	103	67H	g
8	08H	BS	40	28H	(	72	48H	H	104	68H	h
9	09H	HT	41	29H	)	73	49H	I	105	69H	i
10	0AH	LF	42	2AH	*	74	4AH	J	106	6AH	j
11	0BH	VT	43	2BH	+	75	4BH	K	107	6BH	k
12	0CH	FF	44	2CH	,	76	4CH	L	108	6CH	l
13	0DH	CR	45	2DH	-	77	4DH	M	109	6DH	m
14	0EH	SO	46	2EH	.	78	4EH	N	110	6EH	n
15	0FH	SI	47	2FH	/	79	4FH	O	111	6FH	o
16	10H	DLE	48	30H	0	80	50H	P	112	70H	p
17	11H	DC1	49	31H	1	81	51H	Q	113	71H	q
18	12H	DC2	50	32H	2	82	52H	R	114	72H	r
19	13H	DC3	51	33H	3	83	53H	X	115	73H	s
20	14H	DC4	52	34H	4	84	54H	T	116	74H	t
21	15H	NAK	53	35H	5	85	55H	U	117	75H	u
22	16H	SYN	54	36H	6	86	56H	V	118	76H	v
23	17H	TB	55	37H	7	87	57H	W	119	77H	w
24	18H	CAN	56	38H	8	88	58H	X	120	78H	x
25	19H	EM	57	39H	9	89	59H	Y	121	79H	y
26	1AH	SUB	58	3AH	:	90	5AH	Z	122	7AH	z
27	1BH	ESC	59	3BH	;	91	5BH	[	123	7BH	{
28	1CH	FS	60	3CH	<	92	5CH	/	124	7CH	
29	1DH	GS	61	3DH	=	93	5DH	l	125	7DH	}
30	1EH	RS	62	3EH	>	94	5EH	^	126	7EH	~
31	1FH	US	63	3FH	?	95	5FH	—	127	7FH	DEL

六进制数，若不够四位前面补零。

数据宽度

数据宽度：数据在存储器中的存储尺度

所有数据的基本存储单位是字节，一个字节占八位（位是计算机存储的最小单位）

其他存储单位word(字)，dword(双字)32位，qword(八字节)64位

数值的表示范围

byte 2^8

word 2^{16}

dword 2^{32}

qword 2^{64}

字节序

字节顺序

数据存储的方式：大端序，小端序



44属于低位字节，所以在小端模式中被保存在低地址中，从左到右依次以低地址到高地址的顺序保存33，22，11；而大端模式则正好相反。

windows下为小端序

Unix下为大端序

网络传输数据为大端序

ASCII

字符映射成整数

<https://csdnimg.cn/release/blogv2/dist/pc/themesSkin/skin3-template/images/bg.png?v20220120>

编程判断主机字节序

1.取值比较法

先定义一个十六进制数，通过指针取出

```
#include <stdio.h>
#include <windows.h>

int main() {
    DWORD num = 0x01020304;
    if (*(BYTE *)&num == 0x01) {
        printf("大端序\n");
    } else {
        printf("小端序");
    }
    return 0;
}
```

2.直接转换比较法

```
#include<winsock2.h>
```

```
htonl(num)
```

```
#include <stdio.h>
#include <winsock2.h>

int main () {
    DWORD Num = 0x01020304;
    if ( Num == htonl(Num) ) {
        printf("Big Sequence");
    } else {
        printf("Small sequence");
    }
    return 0;
}
```

汇编语言

寄存器

x86体系下的cpu有两种工作状态

- 1.实模式
- 2.保护模式

寄存器：cpu内部用于储存数据的小型存储单元，访问速度比内存高很多

基本寄存器：

- 1.八个通用寄存器

数据寄存器：eax(累加器),ebx(基址寄存器),ecx(计数器),edx(数据寄存器)

eax后16位为AX，AX高八位和低八位分为AH和AL

指针变址寄存器：ebp(栈底),esp(堆栈栈顶寄存器),esi,edi

2.六个段寄存器

预分配的内存区域

cs ss ds es fs gs

3.一个指令指针寄存器

EIP

4.标志寄存器

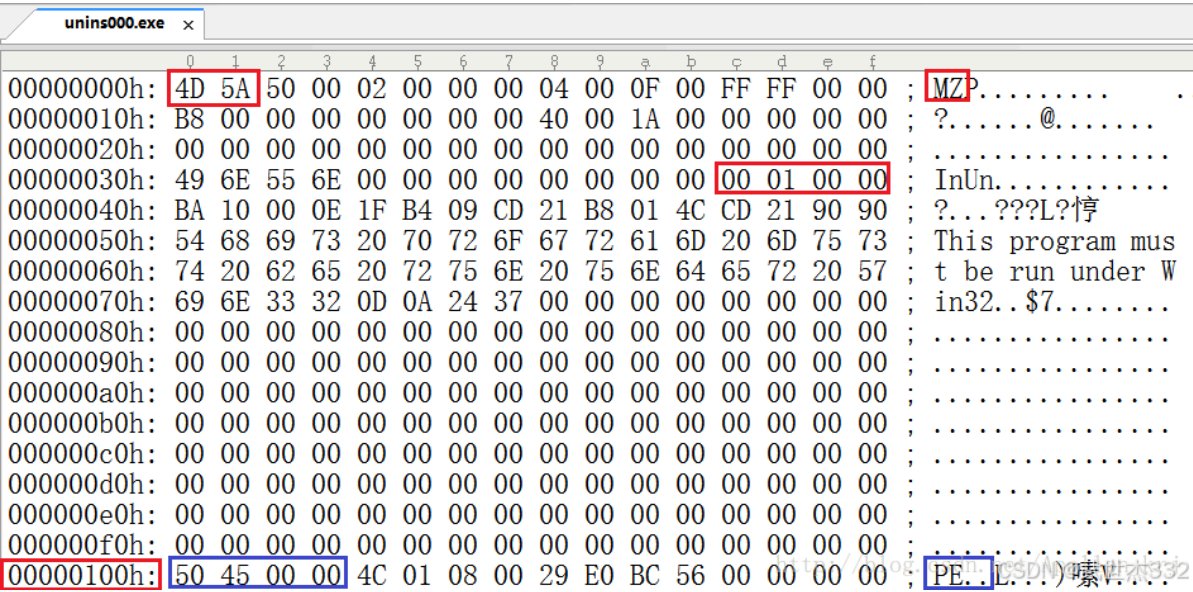
EFLAGS

PE文件初步

一.dos头

PE文件结构：DOS头+PE头+节表+.data/.rdata/.text。

DOS头中声明用的寄存器(我们可以看到e_ss、e_sp、e_ip、e_cs还是16位的寄存器)，所以在32位/64为系统中用到的只有两个成员了（第一个和最后一个）：①e_magic：判断一个文件是不是PE文件；②e_lfanew：相对于文件首的偏移量，用于找到PE头；



64字节（共4行）的DOS头，第一个成员2个字节是可执行文件的标志信息；最后一个成员4字节是PE头的偏移地址为00000100H

二.PE头

根据DOS头的e_lfanew成员我们就可以找到NT头，NT头的第一个成员是“PE\0\0”（0X50 0X45 0X00 0X00四字节的签名，可以在上图00000100H地址处观察）。

后两个成员则分别是标准PE头（_IMAGE_FILE_HEADER）

首先四字节是NT第一个签名成员“PE\0\0”。接着便是2字节的CPU平台信息：014C，即X86平台

第二个成员区块数量0008H即8个块；

第三个成员为时间戳：56BCE029H(即十进制1455218729)，转换成北京时间为2016-02-12 03:25:29（时间戳在线转换）

第四个和第五个成员各占4字节且均为0；

第六个成员占2字节为默认值E0H（即可选PE头的大小为224字节）；

最后一个属性成员占2字节为“818F”。

可选PE头（_IMAGE_OPTIONAL_HEADER）。

00000110h:	00 00 00 00 E0 00 8F 81 0B 01 02 19 00 F8 0F 00	前 30 个 成员 60H
00000120h:	00 94 03 00 00 00 00 00 6C 05 10 00 00 10 00 00	
00000130h:	00 10 10 00 00 00 40 00 00 10 00 00 00 02 00 00	
00000140h:	05 00 00 00 06 00 00 00 05 00 00 00 00 00 00 00	
00000150h:	00 50 14 00 00 04 00 00 3F 17 14 00 02 00 40 81	
00000160h:	00 00 10 00 00 40 00 00 00 00 10 00 00 10 00 00	
00000170h:	00 00 00 00 10 00 00 00 00 00 00 00 00 00 00 00	最后一个 成员：结 构体数组 128字节 (16*8)
00000180h:	00 C0 10 00 40 38 00 00 00 20 11 00 14 25 03 00	
00000190h:	00 00 00 00 00 00 00 00 00 90 13 00 90 16 00 00	
000001a0h:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000001b0h:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000001c0h:	00 10 11 00 18 00 00 00 00 00 00 00 00 00 00 00	
000001d0h:	00 00 00 00 00 00 00 00 80 CA 10 00 8C 08 00 00	
000001e0h:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
000001f0h:	00 00 00 00 00 00 00 00 2E 74 65 78 74 00 00 00	

可选PE头紧接着标准PE头，其大小在标准PE头中给出：大小为E0H即224字节

4个PE首地址字节后数20个字节为标准PE头，在标准PE头后数224个字节是可选PE头